Regression :

1 : Predicting Employee Salaries : Problem Statement: Predicting Employee Salaries Project Description: Build a regression model to predict employee salaries based on factors such as job title, years of experience, education level, and performance metrics. Domain: Human Resources

In [1]:
```
import numpy as np
import pandas as pd
data = pd.read_csv('Salary Data.csv')
data
```

Out[1]:

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
0	32.0	Male	Bachelor's	Software Engineer	5.0	90000.0
1	28.0	Female	Master's	Data Analyst	3.0	65000.0
2	45.0	Male	PhD	Senior Manager	15.0	150000.0
3	36.0	Female	Bachelor's	Sales Associate	7.0	60000.0
4	52.0	Male	Master's	Director	20.0	200000.0
...	...	...	...	...	...	...
370	35.0	Female	Bachelor's	Senior Marketing Analyst	8.0	85000.0
371	43.0	Male	Master's	Director of Operations	19.0	170000.0
372	29.0	Female	Bachelor's	Junior Project Manager	2.0	40000.0
373	34.0	Male	Bachelor's	Senior Operations Coordinator	7.0	90000.0
374	44.0	Female	PhD	Senior Business Analyst	15.0	150000.0

375 rows × 6 columns

In [3]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 375 entries, 0 to 374
Data columns (total 6 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   Age                  373 non-null    float64
 1   Gender               373 non-null    object 
 2   Education Level      373 non-null    object 
 3   Job Title            373 non-null    object 
 4   Years of Experience  373 non-null    float64
 5   Salary               373 non-null    float64
dtypes: float64(3), object(3)
memory usage: 17.7+ KB
```

In [4]:
```
data.isnull().sum()
```

Out[4]:
```
Age                    2
Gender                 2
Education Level        2
Job Title              2
Years of Experience    2
Salary                 2
dtype: int64
```

In [5]:
```
data.dropna(inplace = True)
```

In [6]:
```
data.isna().sum()
```

Out[6]:
```
Age                    0
Gender                 0
Education Level        0
Job Title              0
Years of Experience    0
Salary                 0
dtype: int64
```

In [7]:
```
data.describe()
```

Out[7]:

	Age	Years of Experience	Salary
count	373.000000	373.000000	373.000000
mean	37.431635	10.030831	100577.345845
std	7.069073	6.557007	48240.013482
min	23.000000	0.000000	350.000000
25%	31.000000	4.000000	55000.000000
50%	36.000000	9.000000	95000.000000
75%	44.000000	15.000000	140000.000000
max	53.000000	25.000000	250000.000000

In [8]:
```
data.head(10)
```

Out[8]:

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
0	32.0	Male	Bachelor's	Software Engineer	5.0	90000.0
1	28.0	Female	Master's	Data Analyst	3.0	65000.0
2	45.0	Male	PhD	Senior Manager	15.0	150000.0
3	36.0	Female	Bachelor's	Sales Associate	7.0	60000.0
4	52.0	Male	Master's	Director	20.0	200000.0
5	29.0	Male	Bachelor's	Marketing Analyst	2.0	55000.0
6	42.0	Female	Master's	Product Manager	12.0	120000.0
7	31.0	Male	Bachelor's	Sales Manager	4.0	80000.0
8	26.0	Female	Bachelor's	Marketing Coordinator	1.0	45000.0
9	38.0	Male	PhD	Senior Scientist	10.0	110000.0

In [9]:
```
data.tail(10)
```

Out[9]:

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
365	43.0	Male	Master's	Director of Marketing	18.0	170000.0
366	31.0	Female	Bachelor's	Junior Financial Analyst	3.0	50000.0
367	41.0	Male	Bachelor's	Senior Product Manager	14.0	150000.0
368	44.0	Female	PhD	Senior Data Engineer	16.0	160000.0
369	33.0	Male	Bachelor's	Junior Business Analyst	4.0	60000.0
370	35.0	Female	Bachelor's	Senior Marketing Analyst	8.0	85000.0
371	43.0	Male	Master's	Director of Operations	19.0	170000.0
372	29.0	Female	Bachelor's	Junior Project Manager	2.0	40000.0
373	34.0	Male	Bachelor's	Senior Operations Coordinator	7.0	90000.0
374	44.0	Female	PhD	Senior Business Analyst	15.0	150000.0

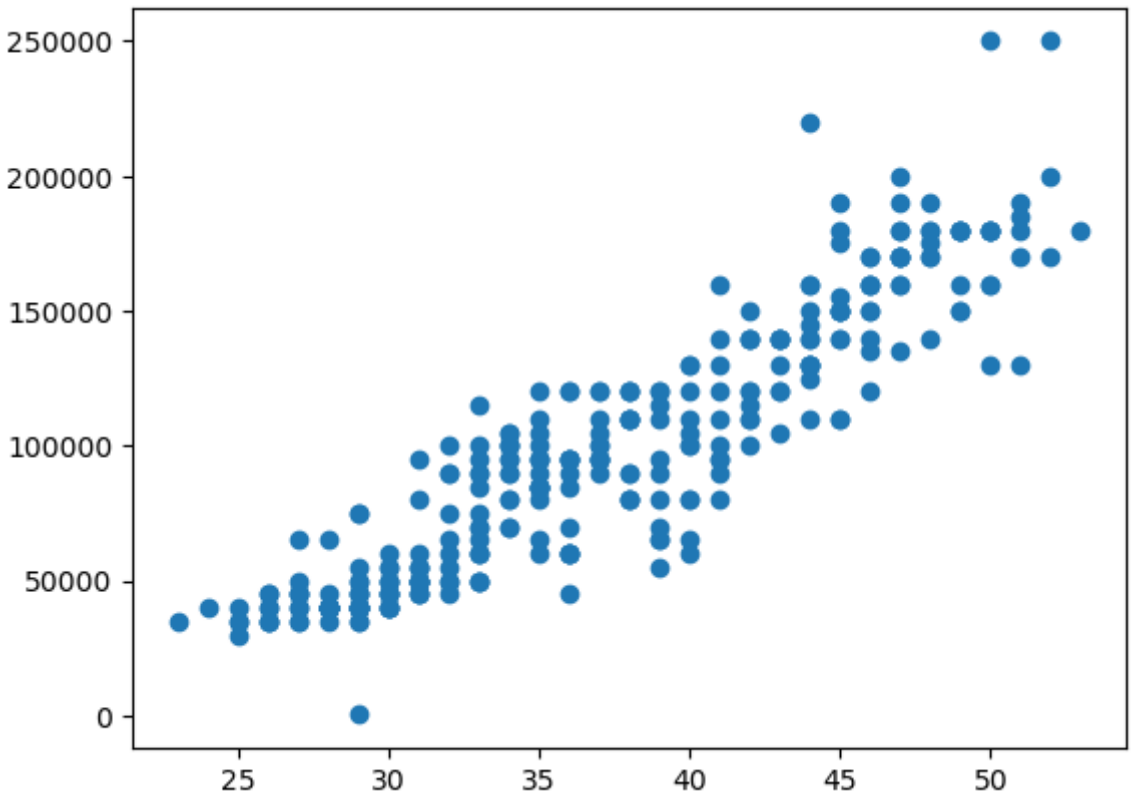
In [10]:
```
data.columns
```

Out[10]:
```
Index(['Age', 'Gender', 'Education Level', 'Job Title', 'Years of Experience',
       'Salary'],
      dtype='object')
```

In [12]:
```
####Plotting Scatter plot to check relationship between independent variable and dependent variable
import matplotlib.pyplot as plt
```

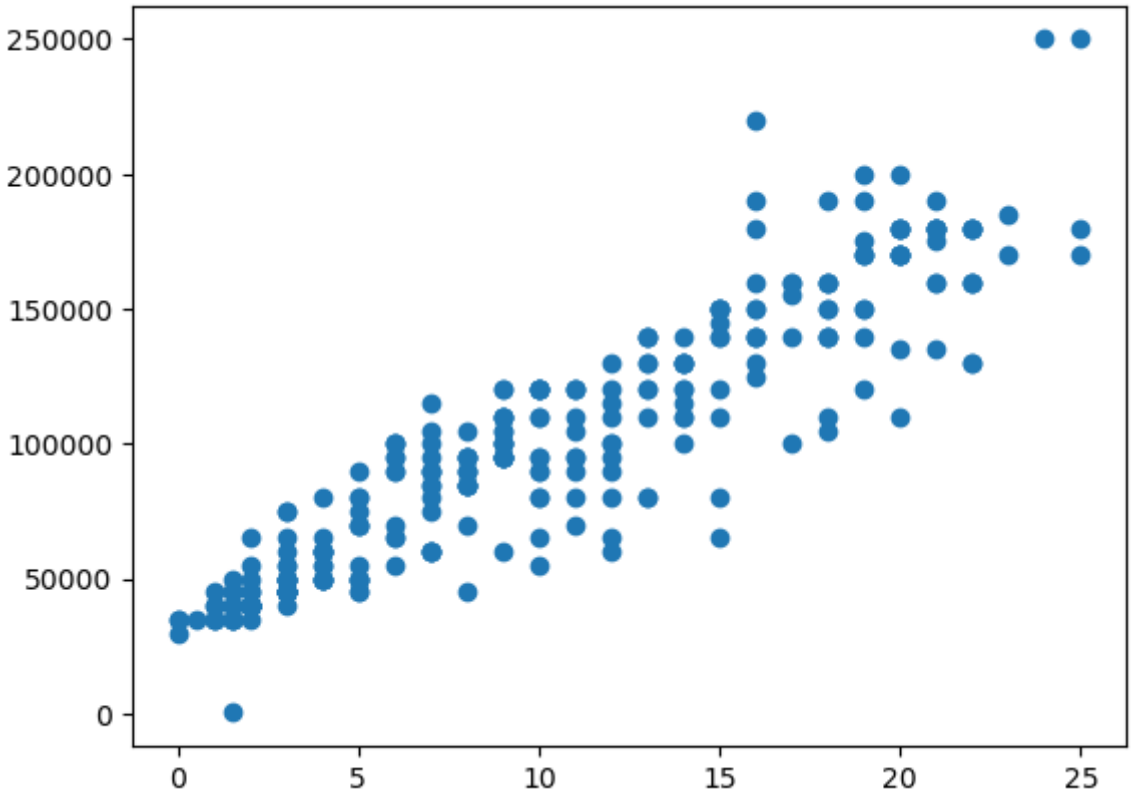
In [13]:
```
plt.scatter(data['Age'][:300], data["Salary"][:300])
```

Out[13]:
```
<matplotlib.collections.PathCollection at 0x27c42f3e040>
```

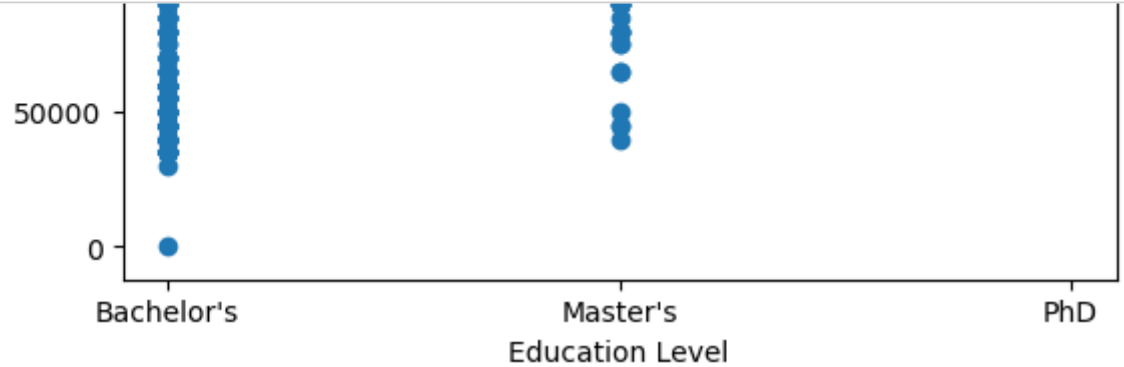


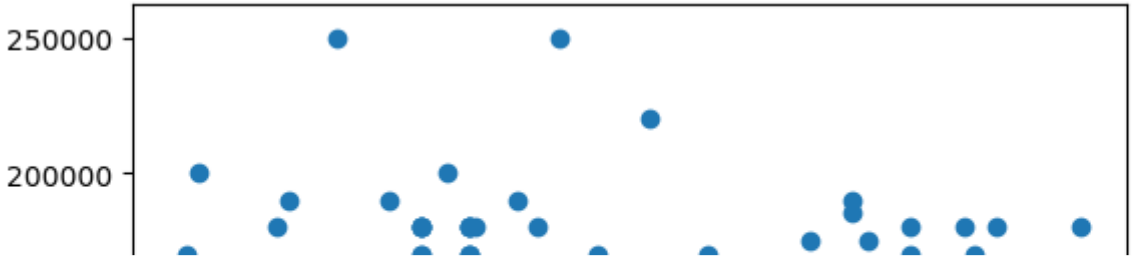
In [14]:
```
plt.scatter(data['Years of Experience'][:300], data["Salary"][:300])
```

Out[14]:
```
<matplotlib.collections.PathCollection at 0x27c438345b0>
```



In [15]:
```
for i in data.columns[:-1]:
    plt.xlabel(i)
    plt.ylabel("Salary")
    plt.scatter(data[i][:300],data["Salary"][:300])
    plt.show()
```





In [16]:
```
x = np.array(data["Years of Experience"]).reshape(-1,1)
y = np.array(data["Salary"]).reshape(-1,1)
```

In [17]:
```
import numpy as np
from sklearn.linear_model import LinearRegression
linear = LinearRegression()
linear.fit(x,y)
```

Out[17]:
```
LinearRegression()
```

In [18]:
```
linear.predict([[48]])
```

Out[18]:
```
array([[360457.72161228]])
```

multiple regression

In [19]:
```
len(data.columns)
```

Out[19]:
```
6
```

In [20]:
```
data.columns
```

Out[20]:
```
Index(['Age', 'Gender', 'Education Level', 'Job Title', 'Years of Experience',
       'Salary'],
      dtype='object')
```

In [31]:
```
#### Import label encoder
from sklearn import preprocessing

label_encoder = preprocessing.LabelEncoder()

data['Gender']= label_encoder.fit_transform(data['Gender'])

data['Gender'].unique()
```

Out[31]:
```
array([1, 0])
```

In [44]:
```
#### Import label encoder
from sklearn import preprocessing

label_encoder = preprocessing.LabelEncoder()

data['Education Level']= label_encoder.fit_transform(data['Education Level'])

data['Education Level'].unique()
```

Out[44]:
```
array([0, 1, 2])
```

In [51]:
```
#### Import label encoder
from sklearn import preprocessing

label_encoder = preprocessing.LabelEncoder()

data['Job Title']= label_encoder.fit_transform(data['Job Title'])

data['Job Title'].unique()
```

Out[51]:
```
array([159,  17, 130, 101,  22,  81,  93, 104,  82, 150, 158,  40,  36,
        96,  13,  89,  83, 116,  18, 102,   3, 172,  44,  98,  37, 157,
       160,  57, 112,  92,   6,   1,  19,  84, 167,  39,  95,  16, 103,
       169,  88,  85,   2, 162,  10,   0,  29,  41,  12,   5,  34, 171,
        38, 170, 156,  30, 115,  47,  21,  43,  14,   4, 118, 173,  99,
       166,  11, 153,  42,   9, 165, 106,   8,  56,  35,  46, 144,  91,
       164, 134, 168, 100,  76,  97,  87,  94, 122,  80, 143,   7,  20,
        45, 131,  15, 120, 161, 163, 110,  62,  86,  90,  61, 141,  66,
       121, 105, 152,  79, 154, 146,  73,  64,  54, 142,  49, 148,  65,
        70, 109,  32,  71, 111, 139,  53, 129,  59, 138,  28,  77, 149,
        31,  52, 132, 125,  50, 108, 147,  60,  25,  63,  55, 136, 124,
       155,  69, 135, 128, 145,  33, 107,  23,  74, 126, 113,  27,  48,
        78, 133, 127, 117,  51,  75, 140,  68, 151,  72, 119, 123, 114,
        67,  26, 137,  58,  24])
```

In [52]:
```
data
```

Out[52]:

	Age	Gender	Education Level	Job Title	Years of Experience	Salary
0	32.0	1	0	159	5.0	90000.0
1	28.0	0	1	17	3.0	65000.0
2	45.0	1	2	130	15.0	150000.0
3	36.0	0	0	101	7.0	60000.0
4	52.0	1	1	22	20.0	200000.0
...	...	...	...	...	...	...
370	35.0	0	0	131	8.0	85000.0
371	43.0	1	1	30	19.0	170000.0
372	29.0	0	0	70	2.0	40000.0
373	34.0	1	0	137	7.0	90000.0
374	44.0	0	2	110	15.0	150000.0

373 rows × 6 columns

In [53]:
```
x = data.drop("Salary",axis=1)

y = data["Salary"]
```

In [54]:
```
x.head()
```

Out[54]:

	Age	Gender	Education Level	Job Title	Years of Experience
0	32.0	1	0	159	5.0
1	28.0	0	1	17	3.0
2	45.0	1	2	130	15.0
3	36.0	0	0	101	7.0
4	52.0	1	1	22	20.0

In [55]:
```
y.head()
```

Out[55]:
```
0     90000.0
1     65000.0
2    150000.0
3     60000.0
4    200000.0
Name: Salary, dtype: float64
```

In [56]:
```
model = LinearRegression()
```

In [57]:
```
model.fit(x,y)
```

Out[57]:
```
LinearRegression()
```

In [58]:
```
model.coef_
```

Out[58]:
```
array([ 2652.29501743,  8322.28165554, 13290.95033734,    27.3298712 ,
        3156.92278531])
```

In [59]:
```
model.intercept_
```

Out[59]:
```
-44282.390111837245
```

In [62]:
```
x.columns
```

Out[62]:
```
Index(['Age', 'Gender', 'Education Level', 'Job Title', 'Years of Experience'], dtype='object')
```

In [63]:
```
model.predict([[3, 5, 2, 5, 16]])
```

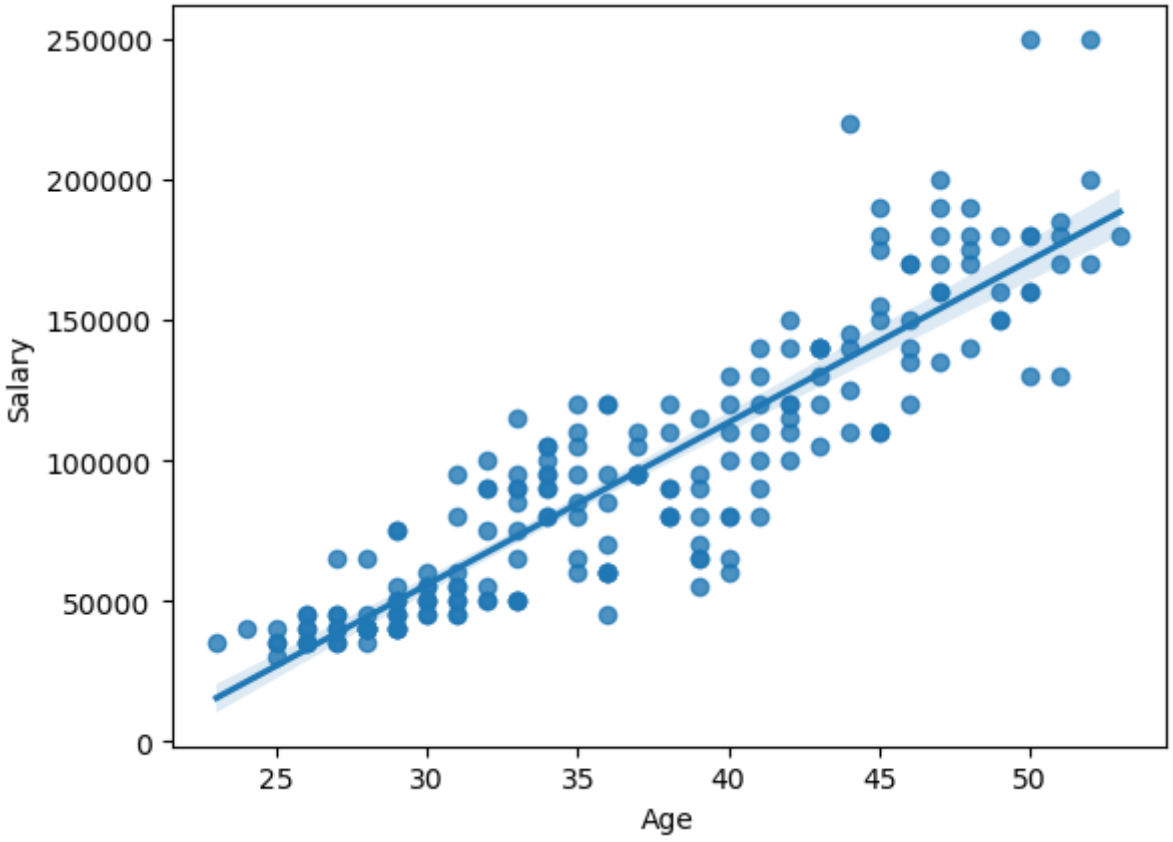
```
C:\Users\admin\anaconda3\lib\site-packages\sklearn\base.py:450: UserWarning: X does not have valid feature names, but
LinearRegression was fitted with feature names
  warnings.warn(
```

Out[63]:
```
array([82515.21781381])
```

In [ ]:
```
Salary and Age
```

In [65]:
```
import seaborn as sns
sns.regplot(x="Age",
            y = "Salary",
            data = data[:200])
```

Out[65]:
```
<AxesSubplot:xlabel='Age', ylabel='Salary'>
```



In [67]:
```
# split the data into testing and training

from sklearn.model_selection import train_test_split
```

In [68]:
```
xtrain, xtest, ytrain , ytest = train_test_split(data.drop("Salary",axis=1),data["Salary"], train_size = .75)
```

In [69]:
```
xtrain.shape
```

Out[69]:
```
(279, 5)
```

In [70]:
```
ytrain.shape
```

Out[70]:
```
(279,)
```

In [71]:
```
xtest.shape
```

Out[71]:
```
(94, 5)
```

In [72]:
```
ytrain.shape
```

Out[72]:
```
(279,)
```

In [73]:
```
xtest.shape
```

Out[73]:
```
(94, 5)
```

In [74]:
```
ytest.shape
```

Out[74]:
```
(94,)
```

In [ ]:
```
model = LinearRegression()
```

In [75]:
```
model.fit(xtrain,ytrain)
```

Out[75]:
```
LinearRegression()
```

In [76]:
```
y_pred = model.predict(xtest)
```

In [78]:
```
ytest.head(10)
```

Out[78]:
```
133    160000.0
196     90000.0
244     70000.0
210    110000.0
118     45000.0
93     170000.0
279    180000.0
173    140000.0
286     60000.0
295     60000.0
Name: Salary, dtype: float64
```

In [79]:
```
xtest.head(10)
```

Out[79]:

	Age	Gender	Education Level	Job Title	Years of Experience
133	47.0	1	2	146	22.0
196	41.0	0	0	132	11.0
244	34.0	0	0	62	6.0
210	38.0	1	1	127	9.0
118	29.0	0	0	61	3.0
93	52.0	0	1	134	25.0
279	49.0	0	1	30	21.0
173	43.0	1	0	144	16.0
286	33.0	1	0	49	4.0
295	33.0	1	0	59	4.0

In [80]:
```
for i in y_pred[:10]:
    print(i)
```

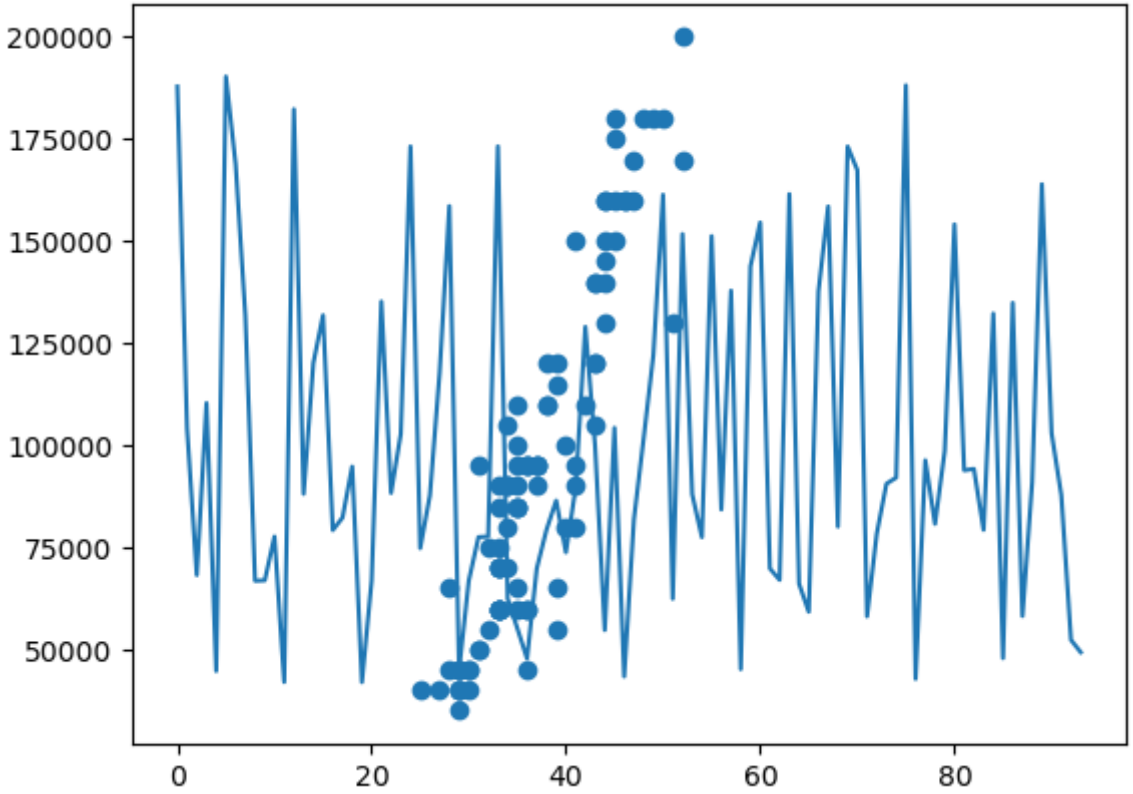
```
187888.09764677595
104165.0623589586
68223.95775895413
110358.44831589288
44716.708794075326
190337.08073629846
168849.51299774245
131861.745183982
66821.59174659693
66933.25283464388
```

In [81]:
```
model.score(xtrain, ytrain )
```

Out[81]:
```
0.9042698954451687
```

In [82]:
```
plt.scatter(xtest["Age"][:100],ytest[:100])
plt.plot(y_pred[:100])
```

Out[82]:
```
[<matplotlib.lines.Line2D at 0x27c46340f10>]
```



In [84]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, mean_squared_log_error
```

In [85]:
```
mean_absolute_error(ytest, y_pred)
```

Out[85]:
```
10962.464182995238
```

In [86]:
```
mean_squared_error(ytest, y_pred)
```
Out[86]:
```
209417535.16259235
```

In [87]:
```
model.score(xtrain,ytrain)
```
Out[87]:
```
0.9042698954451687
```

In [ ]:
```

```

Classification :

4 : Adult Census Income Prediction : Project Title Adult Census Income Prediction Domain Finance

Problem Statement: The Goal is to predict whether a person has an income of more than 50K a year or not. This is basically a binary classification problem where a person is classified into two groups greater than 50K group or less than or equals to 50K group.

In [262]:
```
import numpy as np
import pandas as pd
data1 = pd.read_csv('adult.csv')
data1
```

Out[262]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	country	sala
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=5(
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=5(
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=5(
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=5(
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=5(

In [223]:
```
data1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32561 entries, 0 to 32560
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype 
---  ------          --------------  ----- 
 0   age             32561 non-null  int64 
 1   workclass       32561 non-null  object
 2   fnlwgt          32561 non-null  int64 
 3   education       32561 non-null  object
 4   education-num   32561 non-null  int64 
 5   marital-status  32561 non-null  object
 6   occupation      32561 non-null  object
 7   relationship    32561 non-null  object
 8   race            32561 non-null  object
 9   sex             32561 non-null  object
 10  capital-gain    32561 non-null  int64 
 11  capital-loss    32561 non-null  int64 
 12  hours-per-week  32561 non-null  int64 
 13  country         32561 non-null  object
 14  salary          32561 non-null  object
dtypes: int64(6), object(9)
memory usage: 3.7+ MB
```

In [224]:
```
data1.head()
```

Out[224]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	country	salary
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K

In [225]:
```
data1.dtypes
```

Out[225]:
```
age                int64
workclass         object
fnlwgt             int64
education         object
education-num      int64
marital-status    object
occupation        object
relationship      object
race              object
sex               object
capital-gain       int64
capital-loss       int64
hours-per-week     int64
country           object
salary            object
dtype: object
```

In [226]:
```
data1.describe()
```

Out[226]:

	age	fnlwgt	education-num	capital-gain	capital-loss	hours-per-week
count	32561.000000	3.256100e+04	32561.000000	32561.000000	32561.000000	32561.000000
mean	38.581647	1.897784e+05	10.080679	1077.648844	87.303830	40.437456
std	13.640433	1.055500e+05	2.572720	7385.292085	402.960219	12.347429
min	17.000000	1.228500e+04	1.000000	0.000000	0.000000	1.000000
25%	28.000000	1.178270e+05	9.000000	0.000000	0.000000	40.000000
50%	37.000000	1.783560e+05	10.000000	0.000000	0.000000	40.000000
75%	48.000000	2.370510e+05	12.000000	0.000000	0.000000	45.000000
max	90.000000	1.484705e+06	16.000000	99999.000000	4356.000000	99.000000

In [227]:
```
data1.isnull().sum()
```

Out[227]:
```
age               0
workclass         0
fnlwgt            0
education         0
education-num     0
marital-status    0
occupation        0
relationship      0
race              0
sex               0
capital-gain      0
capital-loss      0
hours-per-week    0
country           0
salary            0
dtype: int64
```

In [228]:
```
data1.dropna(inplace = True)
```

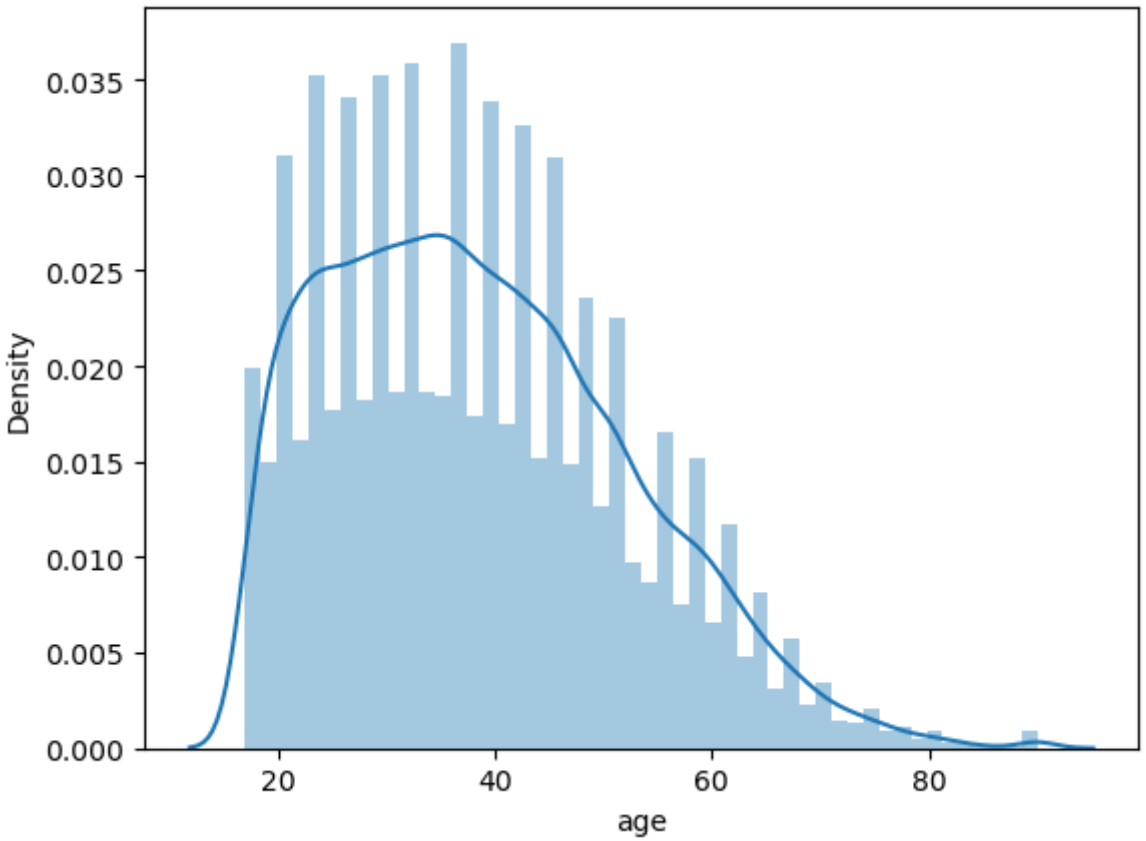
In [229]:
```
data1.isna().sum()
```

Out[229]:
```
age               0
workclass         0
fnlwgt            0
education         0
education-num     0
marital-status    0
occupation        0
relationship      0
race              0
sex               0
capital-gain      0
capital-loss      0
hours-per-week    0
country           0
salary            0
dtype: int64
```

In [230]:
```
import seaborn as sns
sns.distplot(data1.age)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\distributions.py:2619: FutureWarning: `distplot` is a deprecated f
unction and will be removed in a future version. Please adapt your code to use either `displot` (a figure-level funct
ion with similar flexibility) or `histplot` (an axes-level function for histograms).
  warnings.warn(msg, FutureWarning)
```

Out[230]:
```
<AxesSubplot:xlabel='age', ylabel='Density'>
```

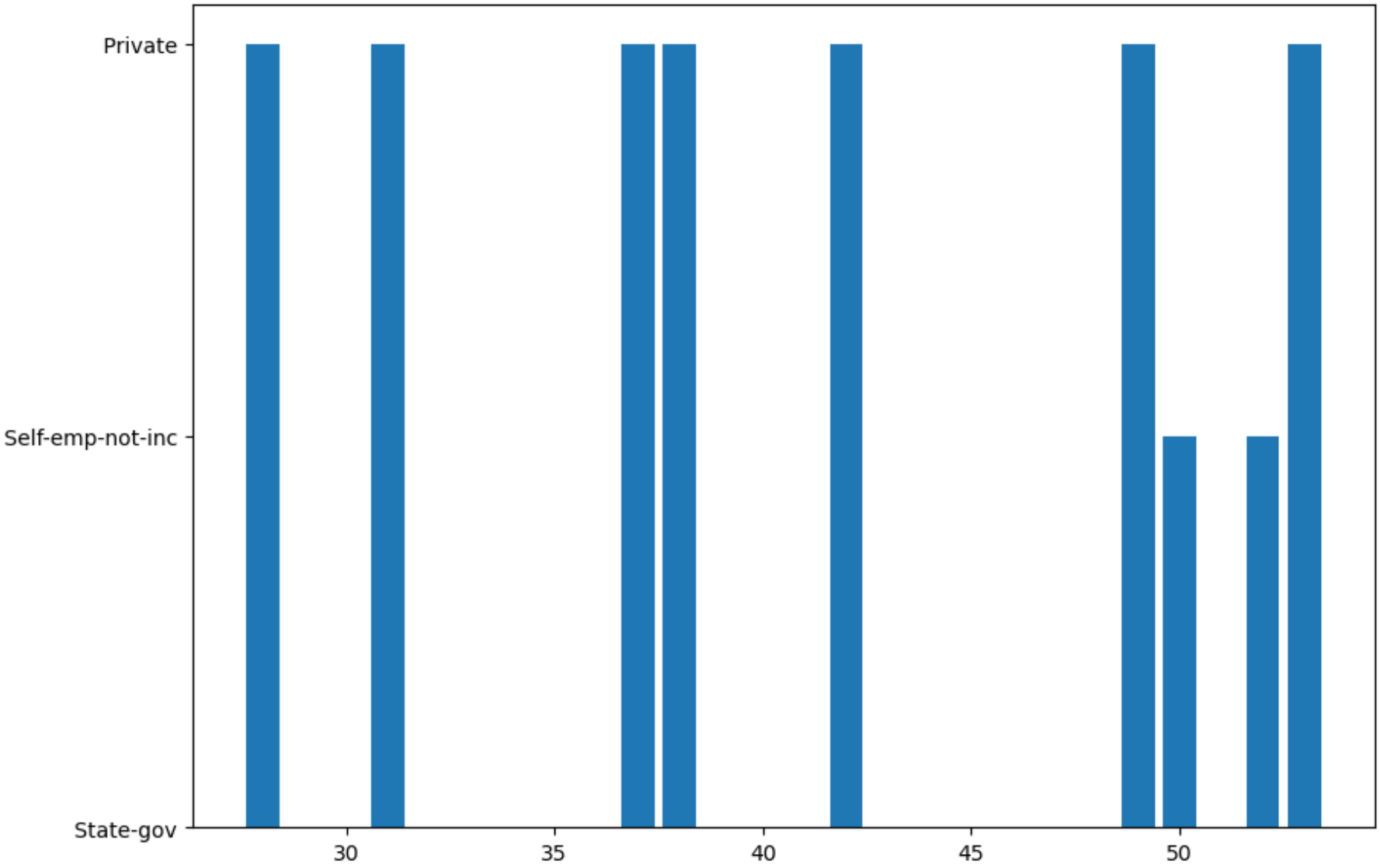


In [231]:
```
data1.workclass.value_counts()
```

Out[231]:
```
Private             22696
Self-emp-not-inc     2541
Local-gov            2093
?                    1836
State-gov            1298
Self-emp-inc         1116
Federal-gov           960
Without-pay            14
Never-worked            7
Name: workclass, dtype: int64
```

In [232]:

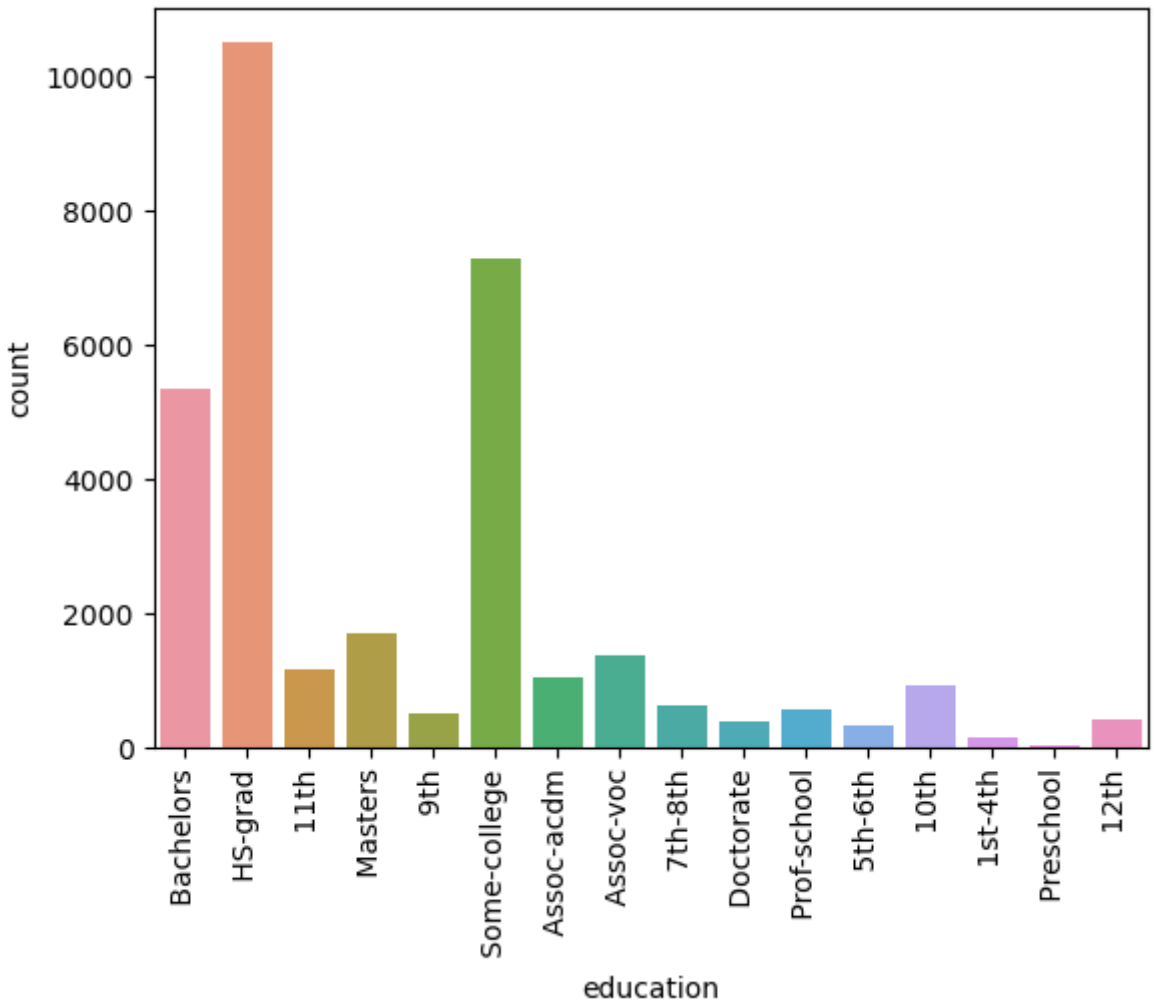
```
name = data1['age'].head(12)
price = data1['workclass'].head(12)

# Figure Size
fig = plt.figure(figsize =(10, 7))

# Horizontal Bar Plot
plt.bar(name[0:10], price[0:10])

# Show Plot
plt.show()
```


Private
Self-emp-not-inc
State-gov
30
35
40
45
50

In [247]:
```
sns.countplot(data1.education)
plt.xticks(rotation = 90)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: FutureWarning: Pass the following variable as a
keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments wit
hout an explicit keyword will result in an error or misinterpretation.
  warnings.warn(
```

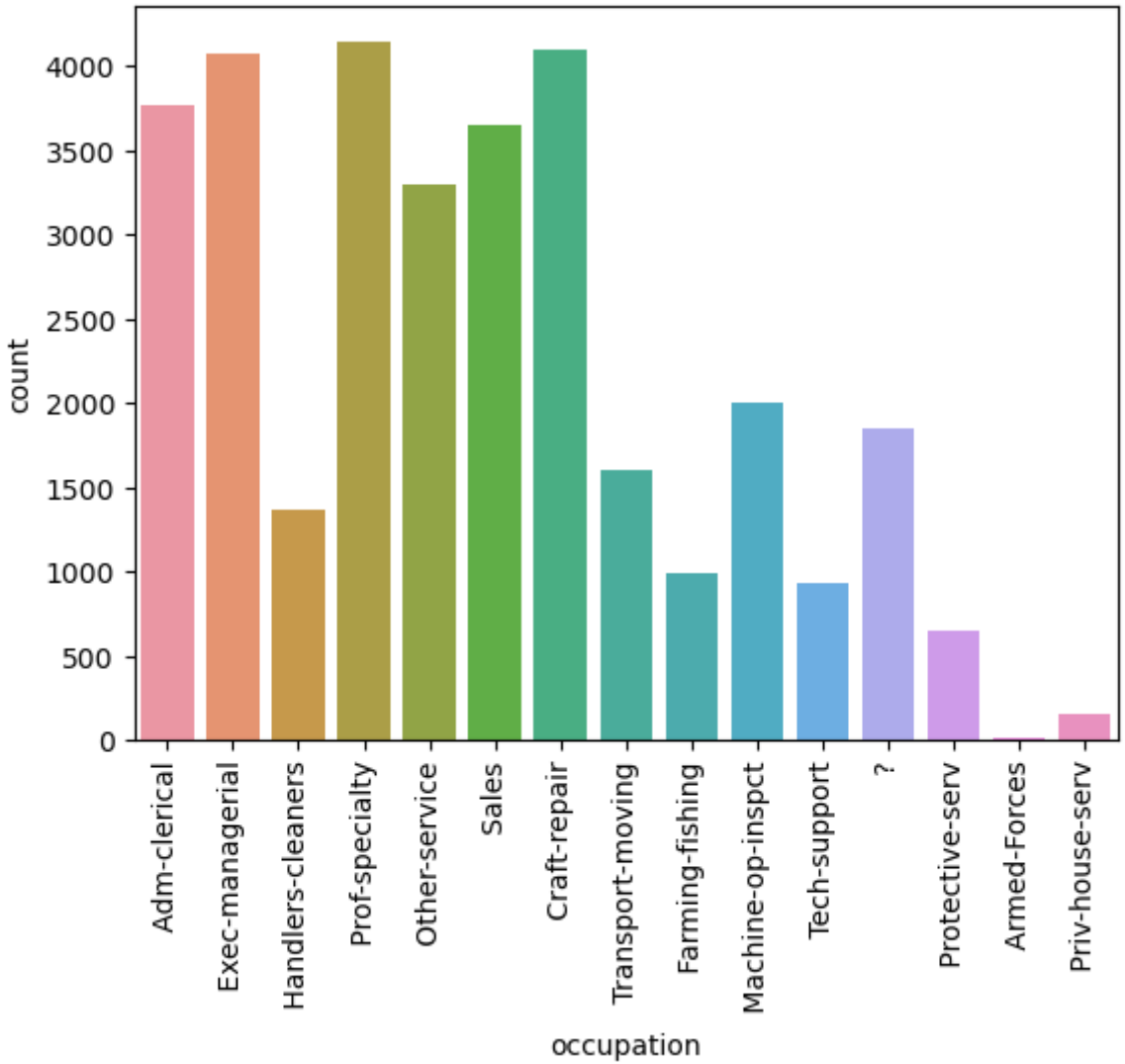
Out[247]:
```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15]),
 [Text(0, 0, ' Bachelors'),
  Text(1, 0, ' HS-grad'),
  Text(2, 0, ' 11th'),
  Text(3, 0, ' Masters'),
  Text(4, 0, ' 9th'),
  Text(5, 0, ' Some-college'),
  Text(6, 0, ' Assoc-acdm'),
  Text(7, 0, ' Assoc-voc'),
  Text(8, 0, ' 7th-8th'),
  Text(9, 0, ' Doctorate'),
  Text(10, 0, ' Prof-school'),
  Text(11, 0, ' 5th-6th'),
  Text(12, 0, ' 10th'),
  Text(13, 0, ' 1st-4th'),
  Text(14, 0, ' Preschool'),
  Text(15, 0, ' 12th')])
```


count
10000
8000
6000
4000
2000
0
Bachelors
HS-grad
11th
Masters
9th
Some-college
Assoc-acdm
Assoc-voc
7th-8th
Doctorate
Prof-school
5th-6th
10th
1st-4th
Preschool
12th
education

In [248]:
```
sns.countplot(data1.occupation)
plt.xticks(rotation = 90)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: FutureWarning: Pass the following variable as a
keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments wit
hout an explicit keyword will result in an error or misinterpretation.
  warnings.warn(
```

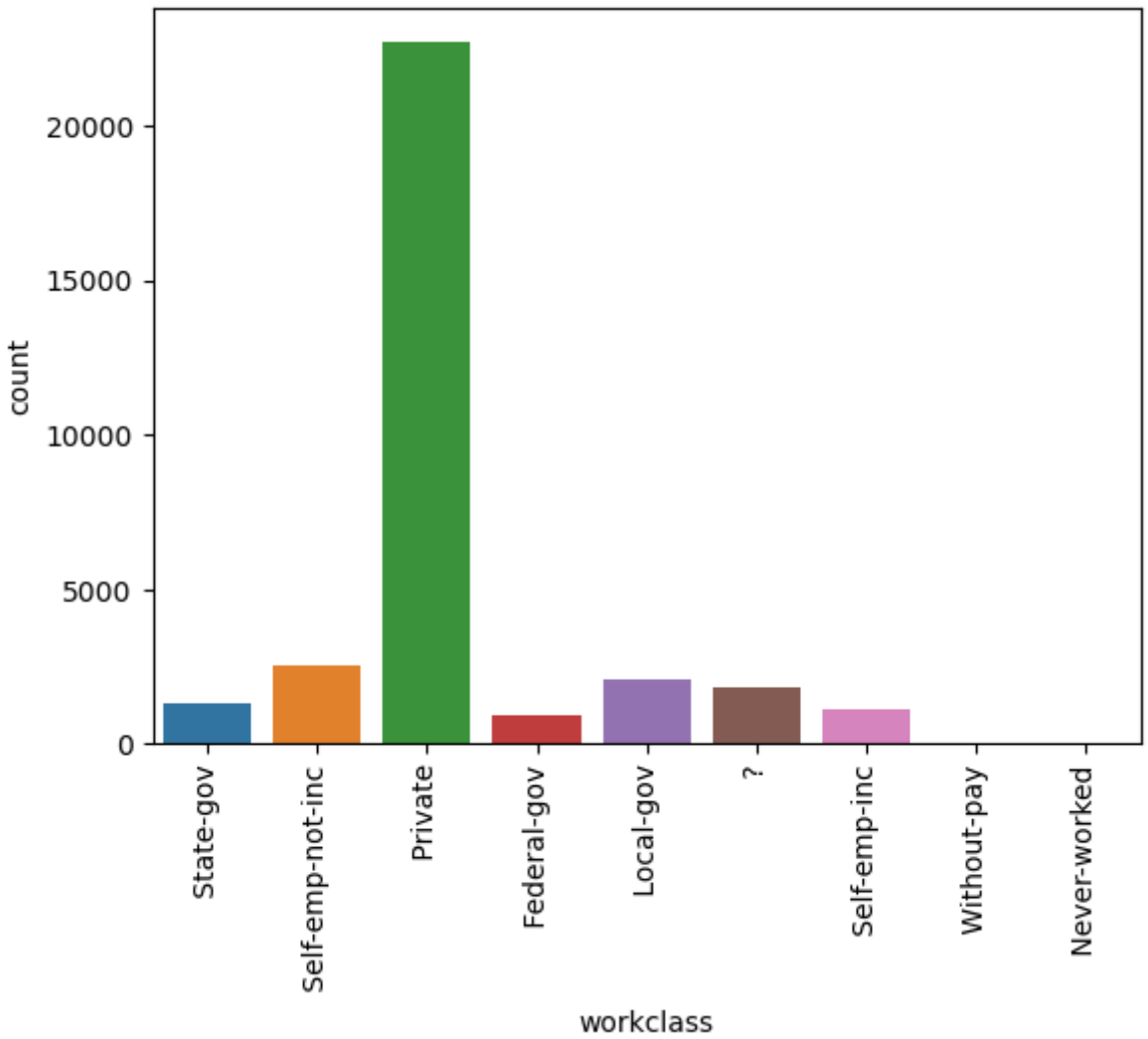
Out[248]:
```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14]),
 [Text(0, 0, ' Adm-clerical'),
  Text(1, 0, ' Exec-managerial'),
  Text(2, 0, ' Handlers-cleaners'),
  Text(3, 0, ' Prof-specialty'),
  Text(4, 0, ' Other-service'),
  Text(5, 0, ' Sales'),
  Text(6, 0, ' Craft-repair'),
  Text(7, 0, ' Transport-moving'),
  Text(8, 0, ' Farming-fishing'),
  Text(9, 0, ' Machine-op-inspct'),
  Text(10, 0, ' Tech-support'),
  Text(11, 0, ' ?'),
  Text(12, 0, ' Protective-serv'),
  Text(13, 0, ' Armed-Forces'),
  Text(14, 0, ' Priv-house-serv')])
```


count
4000
3500
3000
2500
2000
1500
1000
500
0
Adm-clerical
Exec-managerial
Handlers-cleaners
Prof-specialty
Other-service
Sales
Craft-repair
Transport-moving
Farming-fishing
Machine-op-inspct
Tech-support
?
Protective-serv
Armed-Forces
Priv-house-serv
occupation

In [249]:
```
sns.countplot(data1.workclass)
plt.xticks(rotation = 90)
```

```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: FutureWarning: Pass the following variable as a
keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments wit
hout an explicit keyword will result in an error or misinterpretation.
  warnings.warn(
```

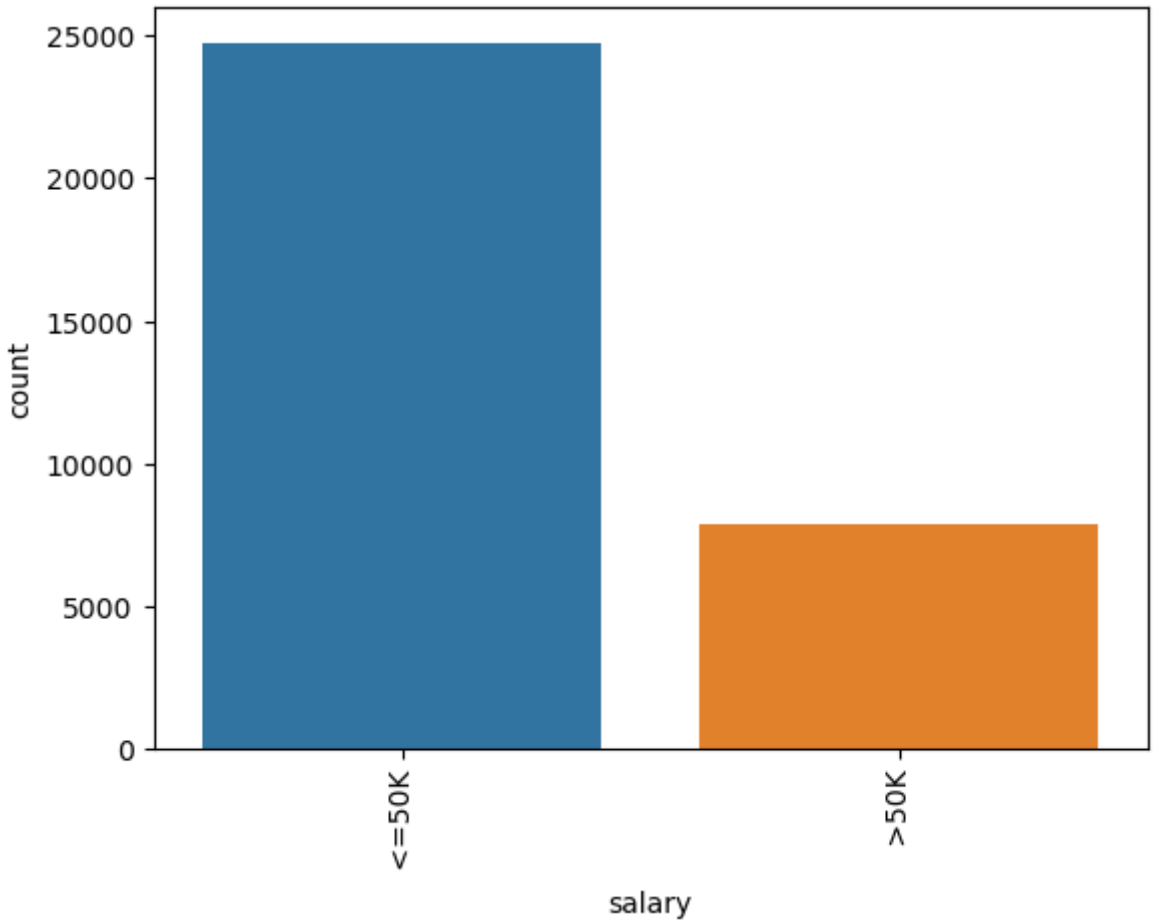
Out[249]:
```
(array([0, 1, 2, 3, 4, 5, 6, 7, 8]),
 [Text(0, 0, ' State-gov'),
  Text(1, 0, ' Self-emp-not-inc'),
  Text(2, 0, ' Private'),
  Text(3, 0, ' Federal-gov'),
  Text(4, 0, ' Local-gov'),
  Text(5, 0, ' ?'),
  Text(6, 0, ' Self-emp-inc'),
  Text(7, 0, ' Without-pay'),
  Text(8, 0, ' Never-worked')])
```


count
20000
15000
10000
5000
0
State-gov
Self-emp-not-inc
Private
Federal-gov
Local-gov
?
Self-emp-inc
Without-pay
Never-worked
workclass

In [250]:
```
sns.countplot(data1.salary)
plt.xticks(rotation = 90)
```

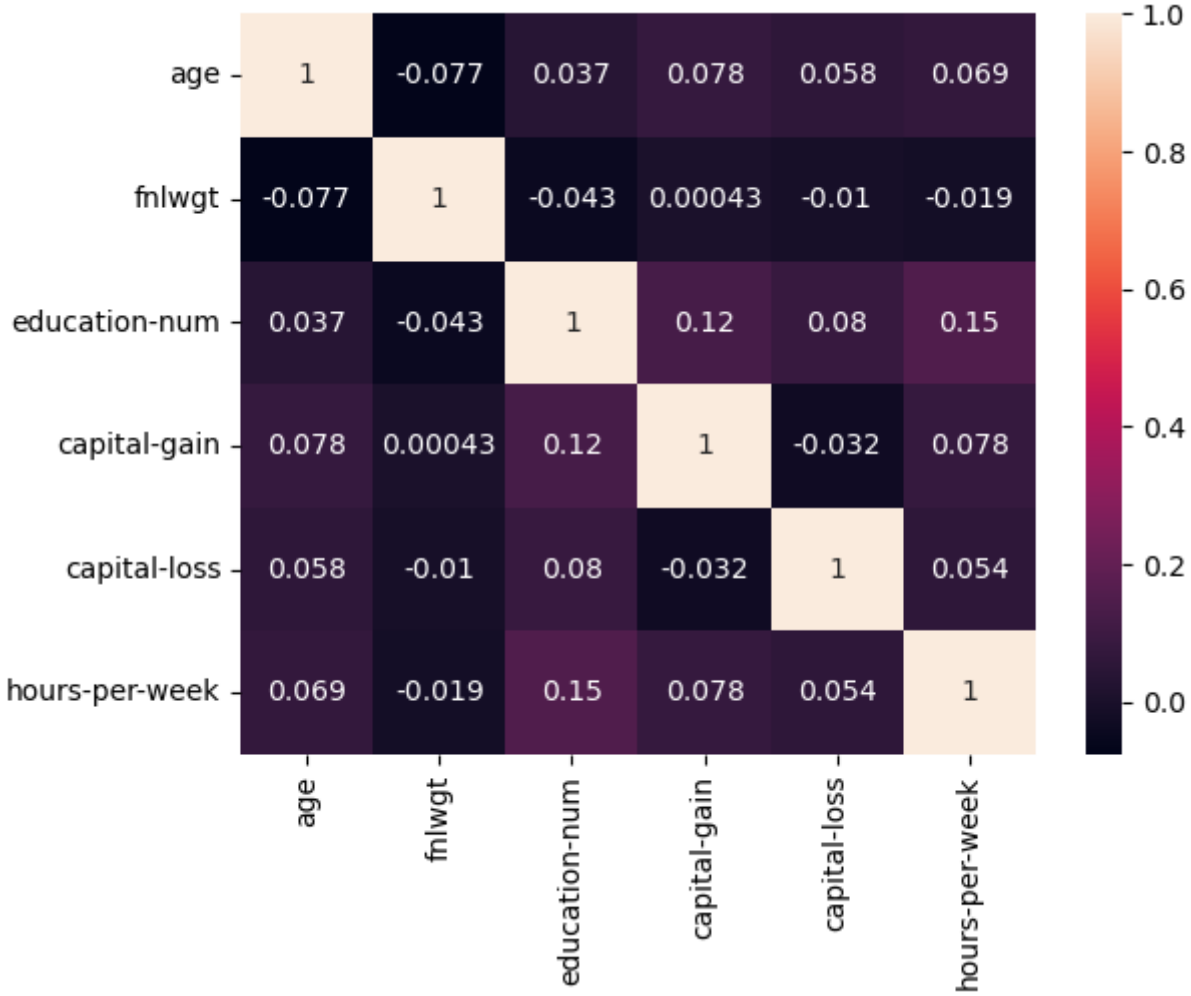
```
C:\Users\admin\anaconda3\lib\site-packages\seaborn\_decorators.py:36: FutureWarning: Pass the following variable as a
keyword arg: x. From version 0.12, the only valid positional argument will be `data`, and passing other arguments wit
hout an explicit keyword will result in an error or misinterpretation.
  warnings.warn(
```

Out[250]:
```
(array([0, 1]), [Text(0, 0, ' <=50K'), Text(1, 0, ' >50K')])
```



In [254]:
```
sns.heatmap(data1.corr(),annot = True)
```

Out[254]: <AxesSubplot:>



In [ ]:

In [ ]:
```
#ENCODING DATA
```

In [266]:
```
#### Import label encoder
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
data1.columns
```

Out[266]:
```
Index(['age', 'workclass', 'fnlwgt', 'education', 'education-num',
       'marital-status', 'occupation', 'relationship', 'race', 'sex',
       'capital-gain', 'capital-loss', 'hours-per-week', 'country', 'salary'],
      dtype='object')
```

In [269]:
```
data1['workclass'] = le.fit_transform(data1['workclass'])
data1['education'] = le.fit_transform(data1['education'])
data1['workclass'] = le.fit_transform(data1['workclass'])

data1['race'] = le.fit_transform(data1['race'])
data1['sex'] = le.fit_transform(data1['sex'])
data1['relationship'] = le.fit_transform(data1['relationship'])
data1['occupation'] = le.fit_transform(data1['occupation'])
data1['country'] = le.fit_transform(data1['country'])
data1['marital-status'] = le.fit_transform(data1['marital-status'])
```

In [270]:
```
data1
```

Out[270]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	country	salary
0	39	7	77516	9	12	4	1	1	4	1	2174	0	40	39	<=50K
1	50	6	83311	9	12	2	4	0	4	1	0	0	13	39	<=50K
2	38	4	215646	11	8	0	6	1	4	1	0	0	40	39	<=50K
3	53	4	234721	1	6	2	6	0	2	1	0	0	40	39	<=50K
4	28	4	338409	9	12	2	10	5	2	0	0	0	40	5	<=50K
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
32556	27	4	257302	7	11	2	13	5	4	0	0	0	38	39	<=50K
32557	40	4	154374	11	8	2	7	0	4	1	0	0	40	39	>50K
32558	58	4	151910	11	8	6	1	4	4	0	0	0	40	39	<=50K
32559	22	4	201490	11	8	4	1	3	4	1	0	0	20	39	<=50K
32560	52	5	287927	11	8	2	4	5	4	0	15024	0	40	39	>50K

32561 rows × 15 columns

In [271]:
```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
```

In [272]:
```
ds_x = data.drop('salary',axis = 'columns')
y = data1['salary']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_3024\3652578982.py in <module>
----> 1 ds_x = data.drop('salary',axis = 'columns')
      2 y = data1['salary']

~\anaconda3\lib\site-packages\pandas\util\_decorators.py in wrapper(*args, **kwargs)
    309                     stacklevel=stacklevel,
    310                 )
--> 311             return func(*args, **kwargs)
    312
    313         return wrapper

~\anaconda3\lib\site-packages\pandas\core\frame.py in drop(self, labels, axis, index, columns, level, inplace, error
s)
   4955                 weight  1.0     0.8
   4956         """
-> 4957         return super().drop(
   4958             labels=labels,
   4959             axis=axis,

~\anaconda3\lib\site-packages\pandas\core\generic.py in drop(self, labels, axis, index, columns, level, inplace, erro
rs)
   4265         for axis, labels in axes.items():
   4266             if labels is not None:
-> 4267                 obj = obj._drop_axis(labels, axis, level=level, errors=errors)
   4268
   4269         if inplace:

~\anaconda3\lib\site-packages\pandas\core\generic.py in _drop_axis(self, labels, axis, level, errors, consolidate, on
ly_slice)
   4309                 new_axis = axis.drop(labels, level=level, errors=errors)
   4310             else:
-> 4311                 new_axis = axis.drop(labels, errors=errors)
   4312             indexer = axis.get_indexer(new_axis)
   4313

~\anaconda3\lib\site-packages\pandas\core\indexes\base.py in drop(self, labels, errors)
   6659         if mask.any():
   6660             if errors != "ignore":
-> 6661                 raise KeyError(f"{list(labels[mask])} not found in axis")
```

```
6662            indexer = indexer[~mask]
6663        return self.delete(indexer)

KeyError: "['salary'] not found in axis"
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [264]:

```
#### Import label encoder
from sklearn import preprocessing

label_encoder = preprocessing.LabelEncoder()

data['education']= label_encoder.fit_transform(data['education'])

data['education'].unique()
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
~\anaconda3\lib\site-packages\pandas\core\indexes\base.py in get_loc(self, key, method, tolerance)
   3628             try:
-> 3629                 return self._engine.get_loc(casted_key)
   3630             except KeyError as err:

~\anaconda3\lib\site-packages\pandas\_libs\index.pyx in pandas._libs.index.IndexEngine.get_loc()

~\anaconda3\lib\site-packages\pandas\_libs\index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas\_libs\hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas\_libs\hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'education'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_3024\1676206214.py in <module>
      4 label_encoder = preprocessing.LabelEncoder()
      5
----> 6 data['education']= label_encoder.fit_transform(data['education'])
      7
      8 data['education'].unique()

~\anaconda3\lib\site-packages\pandas\core\frame.py in __getitem__(self, key)
   3503             if self.columns.nlevels > 1:
   3504                 return self._getitem_multilevel(key)
-> 3505             indexer = self.columns.get_loc(key)
   3506             if is_integer(indexer):
   3507                 indexer = [indexer]

~\anaconda3\lib\site-packages\pandas\core\indexes\base.py in get_loc(self, key, method, tolerance)
   3629                 return self._engine.get_loc(casted_key)
   3630             except KeyError as err:
-> 3631                 raise KeyError(key) from err
   3632             except TypeError:
   3633                 # If we have a listlike key, _check_indexing_error will raise
```

```
KeyError: 'education'
```

In [255]:
```
one_hot_encoded_data = pd.get_dummies(data1, columns = ['workclass','education','marital-status','occupation','relatio
print(one_hot_encoded_data)
```

```
       age  fnlwgt  education-num  capital-gain  capital-loss  hours-per-week  \
0       39   77516             13          2174             0              40
1       50   83311             13             0             0              13
2       38  215646              9             0             0              40
3       53  234721              7             0             0              40
4       28  338409             13             0             0              40
...    ...     ...            ...           ...           ...             ...
32556   27  257302             12             0             0              38
32557   40  154374              9             0             0              40
32558   58  151910              9             0             0              40
32559   22  201490              9             0             0              20
32560   52  287927              9         15024             0              40

       salary  workclass_ ?  workclass_ Federal-gov  workclass_ Local-gov  \
0       <=50K             0                       0                     0
1       <=50K             0                       0                     0
2       <=50K             0                       0                     0
3       <=50K             0                       0                     0
4       <=50K             0                       0                     0
...       ...           ...                     ...                   ...
32556   <=50K             0                       0                     0
32557    >50K             0                       0                     0
32558   <=50K             0                       0                     0
32559   <=50K             0                       0                     0
32560    >50K             0                       0                     0

       ...  country_ Portugal  country_ Puerto-Rico  country_ Scotland  \
0      ...                  0                     0                  0
1      ...                  0                     0                  0
2      ...                  0                     0                  0
3      ...                  0                     0                  0
4      ...                  0                     0                  0
...    ...                ...                   ...                ...
32556  ...                  0                     0                  0
32557  ...                  0                     0                  0
32558  ...                  0                     0                  0
32559  ...                  0                     0                  0
32560  ...                  0                     0                  0

       country_ South  country_ Taiwan  country_ Thailand  \
0                   0                0                  0
```

```
1                   0                0                   0
2                   0                0                   0
3                   0                0                   0
4                   0                0                   0
...               ...              ...                 ...
32556               0                0                   0
32557               0                0                   0
32558               0                0                   0
32559               0                0                   0
32560               0                0                   0

       country_ Trinadad&Tobago  country_ United-States  country_ Vietnam  \
0                             0                       1                 0
1                             0                       1                 0
2                             0                       1                 0
3                             0                       1                 0
4                             0                       0                 0
...                         ...                     ...               ...
32556                         0                       1                 0
32557                         0                       1                 0
32558                         0                       1                 0
32559                         0                       1                 0
32560                         0                       1                 0

       country_ Yugoslavia
0                        0
1                        0
2                        0
3                        0
4                        0
...                    ...
32556                    0
32557                    0
32558                    0
32559                    0
32560                    0

[32561 rows x 109 columns]
```

In [258]:
```
data1
```

Out[258]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relationship	race	sex	capital-gain	capital-loss	hours-per-week	country	salary
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	Husband	White	Male	0	0	13	United-States	<=50K
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	0	0	40	United-States	<=50K
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	Wife	Black	Female	0	0	40	Cuba	<=50K
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
32556	27	Private	257302	Assoc-acdm	12	Married-civ-spouse	Tech-support	Wife	White	Female	0	0	38	United-States	<=50K
32557	40	Private	154374	HS-grad	9	Married-civ-spouse	Machine-op-inspct	Husband	White	Male	0	0	40	United-States	>50K
32558	58	Private	151910	HS-grad	9	Widowed	Adm-clerical	Unmarried	White	Female	0	0	40	United-States	<=50K
32559	22	Private	201490	HS-grad	9	Never-married	Adm-clerical	Own-child	White	Male	0	0	20	United-States	<=50K
32560	52	Self-emp-inc	287927	HS-grad	9	Married-civ-spouse	Exec-managerial	Wife	White	Female	15024	0	40	United-States	>50K

32561 rows × 15 columns

In [197]:
```
import seaborn as sns
sns.distplot(data1.age)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_3024\2835201845.py in <module>
      1 import seaborn as sns
----> 2 sns.distplot(data1.age)

AttributeError: 'numpy.ndarray' object has no attribute 'age'
```

In [192]:
```
import matplotlib.pyplot as plt

for col_name in data1.columns.values:
     print(col_name)
data1[col_name].hist()
plt.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_3024\1075882881.py in <module>
      1 import matplotlib.pyplot as plt
      2
----> 3 for col_name in data1.columns.values:
      4      print(col_name)
      5 data1[col_name].hist()

AttributeError: 'numpy.ndarray' object has no attribute 'columns'
```

In [ ]:
```

```

In [126]:
```
from sklearn.linear_model import LogisticRegression
```

In [127]:
```
log = LogisticRegression()
```

In [128]:
```
log.fit(x,y)
```

```
C:\Users\admin\anaconda3\lib\site-packages\sklearn\linear_model\_logistic.py:814: ConvergenceWarning: lbfgs failed to
converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html (https://scikit-learn.org/stable/modules/preprocessin
g.html)
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression (https://scikit-learn.org/stable/mo
dules/linear_model.html#logistic-regression)
  n_iter_i = _check_optimize_result(
```

Out[128]:
```
LogisticRegression()
```

In [131]:
```
log.intercept_
```

Out[131]:
```
array([ 0.00804428, -0.0006434 ,  0.08631712,  0.11403329,  0.07329346,
        0.04938998,  0.03443316,  0.01038424,  0.02061486,  0.01467391,
        0.02412616,  0.00905034,  0.00955903,  0.01991039,  0.00868598,
       -0.00305208, -0.01249416, -0.01777697, -0.00652993, -0.01153084,
       -0.00751214, -0.04795673, -0.02299084, -0.0408849 , -0.01581137,
       -0.05993571, -0.00934994, -0.05728051, -0.03086118, -0.01059301,
       -0.04865981, -0.00809539, -0.03138252, -0.02151933, -0.001874  ,
       -0.01578144])
```

In [132]:
```
log.coef_
```

Out[132]:
```
array([[ 0.08200942,  0.02074502, -0.00950263,  0.00250368, -0.14523624],
       [-0.32882583,  0.01519892, -0.01284435,  0.11438805, -0.24508527],
       [ 0.33532949,  0.08517073, -0.07767131,  0.00970018, -1.36619145],
       [ 0.40674103,  0.01697657, -0.12991454,  0.01509742, -1.99277566],
       [ 0.28648525, -0.0722164 , -0.06475083,  0.0283074 , -1.13422426],
       [ 0.33525164, -0.05307957, -0.11944379,  0.01077449, -1.14142337],
       [ 0.25650493, -0.01741717, -0.09539205,  0.00832984, -0.67302534],
       [ 0.2731268 ,  0.11348061, -0.16307582,  0.00718995, -0.65963451],
       [ 0.11759024, -0.00829461, -0.04923964,  0.0356223 , -0.30855046],
       [ 0.19099494, -0.04232522, -0.10791559,  0.01799665, -0.41688417],
       [ 0.18097195,  0.07887473,  0.0210966 , -0.0048146 , -0.33265903],
       [ 0.1315799 , -0.09134678, -0.03456164,  0.0196081 , -0.13336853],
       [ 0.05792012, -0.07677091, -0.10207256,  0.0448803 , -0.14305871],
       [ 0.13920246,  0.06600493, -0.044595  ,  0.03561862, -0.30929548],
       [ 0.06014145,  0.04235302, -0.11634275,  0.05144703, -0.16080462],
       [-0.11711508, -0.02642103, -0.05453033,  0.07394244,  0.17388414],
       [-0.10033063,  0.04643635,  0.03908364,  0.05284233,  0.29354193],
       [-0.07748337,  0.00405479, -0.08641877,  0.05466872,  0.28726857],
       [-0.04745368,  0.04382161,  0.08539107,  0.04088091,  0.16256527],
       [-0.03647811, -0.03785295, -0.02415465,  0.05023884,  0.20687358],
       [-0.03032911,  0.01306921, -0.01964768,  0.02248944,  0.15172595],
       [-0.21035353,  0.03692349, -0.279785  ,  0.06427158,  0.60947534],
       [-0.19923183,  0.02022616, -0.01122397,  0.05417554,  0.51711089],
       [-0.25017042, -0.09004438,  0.1844863 ,  0.06016622,  0.73115348],
       [-0.26020027, -0.02605046,  0.04634567,  0.09601432,  0.10515173],
       [-0.24255606, -0.09937004,  0.42467394,  0.06075594,  0.70609185],
       [-0.03862634,  0.01196196,  0.04718865,  0.02612948,  0.19276251],
       [-0.38978014, -0.09652454,  0.3510083 ,  0.06146638,  1.08235192],
       [-0.24947795,  0.15869675,  0.08002251,  0.02580293,  0.89367517],
       [-0.02692165,  0.02593287,  0.01304285, -0.00364594,  0.32861108],
       [-0.25809274, -0.1156738 ,  0.14178951,  0.01796841,  0.94799631],
       [ 0.04893506, -0.0212937 ,  0.03858739, -0.1175337 ,  0.25109579],
       [-0.13292552,  0.01634133,  0.05631732,  0.03816796,  0.49020303],
       [-0.07475122,  0.02239824,  0.01528947,  0.0282309 ,  0.34048339],
       [ 0.12212274,  0.01521663,  0.04594582, -0.38487106,  0.15323546],
       [ 0.04619603,  0.02079764,  0.01281387, -0.71881102,  0.53695968]])
```

In [186]:
```
# Creating a pie chart for 'Marital status'
marital = data1['marital.status'].value_counts()

plt.style.use('default')
plt.figure(figsize=(10, 7))
plt.pie(marital.values, labels=marital.index, startangle=10, explode=(
    0, 0.20, 0, 0, 0, 0, 0), shadow=True, autopct='%1.1f%%')
plt.title('Marital distribution', fontdict={
          'fontname': 'Monospace', 'fontsize': 20, 'fontweight': 'bold'})
plt.legend()
plt.legend(prop={'size': 7})
plt.axis('equal')
plt.show()
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_3024\1392859286.py in <module>
      1 # Creating a pie chart for 'Marital status'
----> 2 marital = data1['marital.status'].value_counts()
      3
      4 plt.style.use('default')
      5 plt.figure(figsize=(10, 7))

IndexError: only integers, slices (`:`), ellipsis (`...`), numpy.newaxis (`None`) and integer or boolean arrays are v
alid indices
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

Unsupervised :

---

5 : Employee Performance Analysis : Employee Performance Analysis: Analyze employee performance data to identify performance patterns or clusters. Procedure: Apply K-means on employee performance metrics, such as productivity, attendance, feedback scores, or goal achievements. Domain: Human Resources

In [3]:
```
import numpy as np
import pandas as pd
data2 = pd.read_csv('Test.csv')
data2
```

Out[3]:

	Employee_ID	Gender	Age	Education_Level	Relationship_Status	Hometown	Unit	Decision_skill_possess	Time_of_service	Time_since
0	EID_22713	F	32.0	5	Single	Springfield	R&D	Conceptual	7.0	
1	EID_9658	M	65.0	2	Single	Lebanon	IT	Directive	41.0	
2	EID_22203	M	52.0	3	Married	Springfield	Sales	Directive	21.0	
3	EID_7652	M	50.0	5	Single	Washington	Marketing	Analytical	11.0	
4	EID_6516	F	44.0	3	Married	Franklin	R&D	Conceptual	12.0	
...	...	...	...	...	...	...	...	...	...	
2995	EID_22547	F	32.0	3	Single	Franklin	Sales	Directive	3.0	
2996	EID_10066	F	NaN	2	Single	Franklin	Marketing	Conceptual	10.0	
2997	EID_7126	F	60.0	4	Single	Lebanon	Logistics	Analytical	35.0	
2998	EID_4929	F	51.0	1	Married	Springfield	IT	Behavioral	23.0	
2999	EID_24750	F	45.0	1	Single	Franklin	Sales	Directive	12.0	

3000 rows × 23 columns

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: